LOCAL-FIRST DEVELOPER WORKSPACE · ENGINEERING REPORT

H

PROJECT REPORT · NUMBER 1

Helpyard
Local PC Control Dashboard

Project overview, system architecture, interface guide, implemented capabilities, verification evidence, risk assessment, and a staged path to a safe release.

Updated: 17 October 2026
Current runtime: HTML/CSS/JavaScript dashboard + Python standard-library bridge
Report status: local prototype; public release not approved

Prepared from the project files and local checks available in this workspace. Capabilities are labelled as implemented, partially implemented, or planned; untested environments are not presented as verified.

Executive summary

Helpyard is a local workstation dashboard built as a single HTML/CSS/JavaScript page paired with a Python standard-library HTTP bridge. The bridge serves the dashboard, connects browser authentication automatically for one launch, and delegates Windows-specific operations to fixed PowerShell scripts.

1
Single-page frontend, no frontend build step

12
Navigation sections currently shown

20+
Local HTTP route patterns in the bridge

What is verified in the current implementation: the Python bridge compiles and its six local HTTP tests pass for authentication, single-use bootstrap, route behavior, approval gates, input limits, and ping responsiveness during slow Ollama chat. The dashboard's 12 navigation targets, 10 API route references, 78 DOM IDs, and JavaScript syntax pass static checks. A live browser check also verified all 12 sections, automatic connection, Windows metrics/processes/services, project discovery, 15 Ollama models, and a `READY` local chat response.

Release decision: suitable for controlled local evaluation, not public internet exposure. The bridge can perform named Windows operations, but arbitrary command execution and MCP startup are disabled. Browser matrix testing, mutation testing, and full security review remain outstanding.

At a glance

Dimension	Current assessment	Evidence / interpretation
Architecture	**Implemented**	One HTML/CSS/JS UI and a localhost-bound Python bridge; PowerShell performs fixed Windows operations; Ollama is optional.
Core local use	**Verified on one workstation**	Python API tests, dashboard checks, one-click launcher startup, and live browser data routes passed.
AI	**Live local chat verified**	15 installed Ollama models were listed and a short chat returned `READY`.
Project intelligence	**Basic discovery**	Lists candidate folders and immediate Git repositories; not a repository analyzer/indexer.
Security posture	**Local boundary present; hardening needed**	Loopback binding, one-time bootstrap, token and origin checks exist; arbitrary PowerShell execution is disabled.
Public readiness	**Not ready**	Do not publish a build connected to the workstation bridge.

Contents

1. Project purpose, scope, and user value
2. Architecture and request/data flow
3. Graphical dashboard walkthrough
4. Feature inventory and implementation status
5. Bridge/API reference and operational behaviour
6. Security, privacy, and trust boundaries

7. Verification performed and evidence
8. Known limitations and prioritized recommendations
9. Upgrade roadmap and acceptance gates
10. Installation, startup, and daily-use guide
11. Appendix: file inventory and glossary

1. Project purpose, scope, and user value

Purpose

Helpyard consolidates common workstation and developer operations in a navigable browser dashboard. The interface is intentionally a regular HTML document; a local Python bridge serves the page and API, and delegates fixed system operations to Windows PowerShell subprocesses.

Intended audience and use cases

- A Windows developer who wants a quick view of CPU, memory, uptime, disks, processes, and services.
- A user who wants to browse a lightweight list of likely project folders and Git repositories.
- A local-AI user who wants to select an installed Ollama model and chat without sending that chat to a cloud provider.
- An operator who needs a browser UI for a small, reviewed set of Windows actions and service controls.

Design principles visible in the implementation

Local-first

Bridge listener binds to `127.0.0.1`; system APIs are served by the same machine. Ollama defaults to loopback.

Single-file UI

HTML, styles, and frontend JavaScript are together in `pc-control-dashboard.html`; no frontend package manager is required.

Visible controls

Sidebar navigation, connection badge, admin status, confirmation prompts, terminal output, and toast feedback expose operational state.

Incremental product direction

The upgrade plan prioritizes testability and safety before larger project indexing, editing, 3D visuals, or public hosting.

> **Scope boundary:** "Helpyard" is currently a workstation dashboard and a foundation for a developer workspace. Several navigation labels imply future integrations, but not all of them are connected to live backends yet. This report differentiates real behavior from display scaffolding.

2. Architecture and request/data flow

The design has three primary runtime components. The browser owns presentation and user interaction; the Python bridge serves the UI and API, enforces local authentication and approval checks, and starts fixed Windows PowerShell scripts; optional Ollama handles local inference. An optional cloud chat path calls provider APIs directly from the browser.

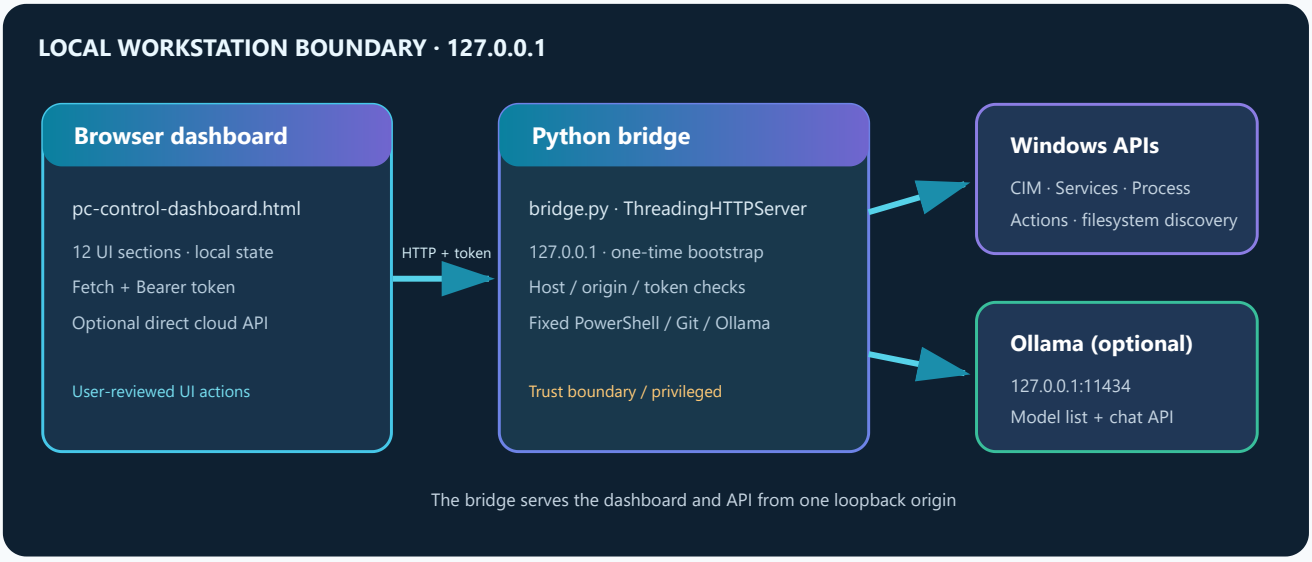


Figure 1. Logical architecture. The browser is not a security boundary for commands; authorization and permissions must be enforced by the bridge.

Typical request lifecycle

1 · User selects panel or command › **2 · Browser** sends HTTP + bearer token › **3 · Bridge** checks local host/origin + token › **4 · Windows / Ollama** executes or returns data › **5 · Dashboard** renders result or error

Runtime responsibilities

Component	Owns	Does not provide
Dashboard HTML	Layout, navigation, rendering, browser state, direct cloud chat requests, HTTP client.	Unrestricted filesystem or OS capability by itself; durable secure credential storage.
Python bridge	Dashboard/API serving, loopback authentication, Windows host metrics, reviewed action/service/process operations, Ollama proxy, Claude Desktop config access.	Remote-safe multi-user authorization, arbitrary command execution (disabled), persistent audit management.
Ollama	Installed local model listing and inference on the local machine.	Guaranteed model availability, latency, or quality; model setup is external.
PowerShell subprocess	Runs fixed scripts for Windows CIM, services, processes, and named actions.	Browser-supplied arbitrary scripts are not passed to PowerShell.

3. Graphical dashboard walkthrough

The live interface uses a dark navy palette, cyan/violet accent gradients, bordered cards, rounded status badges, a sticky navigation rail on wide screens, and a stacked layout at smaller widths. The following is a report illustration of the page structure and expected information hierarchy; it is a designed diagram, not a captured screenshot.



Figure 2. Illustrative overview screen. Actual live values are populated by bridge APIs; the drawing is explanatory and is not evidence of a particular machine reading.

Navigation map

Group	Screen	Primary user task
System	Overview	Review CPU/RAM, uptime, process count, disks, local model availability, tooling labels.
System	Projects	Refresh the bridge-discovered folder and Git repository cards.
System	Workbench	Edit and save assistant system instructions; see ready action cards.
System	Processes	Filter the top process list and request process termination with confirmation.
System	Services	Filter services and request start/restart/stop operations.
System	Terminal	Submit a PowerShell command and read its captured output.
AI	Local AI	Choose a model, refresh installed models, send local chat messages.
AI	Cloud AI	Select Anthropic/OpenAI/Gemini, set model and API key, chat directly with provider.
AI	MCP	View descriptive integration cards; live MCP server management is not wired to these cards.
AI	Skills	Run predefined PowerShell snippets through the same command endpoint.
Security	Security	See static posture/audit labels; this screen is not a live audit engine.
Admin	Settings	Set bridge URL/token; clear saved browser data.

Responsive and visual behaviour

- On wide screens the dashboard uses a 250-pixel sidebar and a flexible main area.
- At viewport widths at or below 900 pixels the sidebar moves above the page content.
- Cards use responsive auto-fit grids; tables are wrapped for horizontal overflow.

- The theme control changes the document background and browser color scheme. Some component styles remain based on the dark palette, so light-mode consistency should be checked in a dedicated visual pass.

4. Feature inventory and implementation status

Status definitions: **Live** means backed by a real local service route and exercised in the tested setup; **Partial** means implemented with important limitations or not fully tested; **UI only** means a label/card is present without the implied complete integration.

Capability	Status	What it currently does	Important boundary
System overview	**Live / verified**	Reads OS/CPU/RAM/uptime/disks/process count, renders rings and CPU/RAM history; failures are surfaced in the page notice.	Verified on one Windows workstation.
Project discovery	**Partial**	Lists the project folder and immediate child Git repositories.	Not a user-granted workspace picker, deep crawler, dependency graph, or source index.
Workbench	**Partial**	Stores a context label and system prompt in browser state; renders predefined action cards.	Does not generate/track an implementation plan or gate every arbitrary command by the prompt.
Process manager	**Live / verified**	Fetches up to 60 processes sorted by working set; filters list; asks for explicit approval before stopping.	Read/list/filter verified; process termination was not exercised.
Service manager	**Live / verified**	Lists Windows services and supports explicitly approved start/restart/stop requests.	Service listing verified; mutations were not exercised.
Terminal	**Disabled by policy**	UI explains that arbitrary command execution is unavailable; the bridge returns HTTP 403 for `/api/run`.	Named, reviewed terminal tasks are not implemented.
Local AI	**Live / verified**	Lists installed Ollama models and sends non-streaming conversation turns through the bridge.	One short chat and model listing verified; requires a local service/model.
Cloud AI	**Partial**	Browser directly calls Anthropic, OpenAI, or Gemini APIs.	Provider calls need the user's key and network access; the key is not persisted by current dashboard state.
MCP	**Disabled / placeholder**	Dashboard describes the disabled state; bridge rejects MCP startup until a reviewed registry exists.	No server registry or active process lifecycle is implemented.
Skills	**Planned placeholders**	Displays example named tasks without executing them.	Command execution and unreviewed snippets remain disabled.
Security / audit	**Static display**	Shows explanatory security and audit status labels.	Labels are hard-coded UI content; there is no persistent action audit log or scan implementation behind them.
Claude Desktop config	**Bridge API only**	Bridge reads/writes the per-user Claude Desktop JSON config and backs up before write.	No corresponding configuration editor is exposed in current dashboard navigation.

> **Interpret "security posture" carefully.** The dashboard's static checklist is descriptive, not the result of an automated audit. It must not be used to claim that the workstation or application passed a security scan.

5. Bridge/API reference

Connection and authentication

- Default bridge port: `8787`. If busy, the Python server probes a short configured list of alternatives.
- Listener prefix: `http://127.0.0.1:<port>/`.
- A random bearer token is generated in memory at startup. The browser redeems a 60-second, single-use bootstrap code supplied in the URL fragment.
- Host, remote IP, and non-null origin are checked for local request patterns; protected API routes call the shared authorization function.
- Ollama base URL defaults to `http://127.0.0.1:11434` and can be overridden with a script parameter.

Route inventory

Method + route	Purpose / data	Impact
`GET /, /pc-control-dashboard.html`	Serve dashboard HTML from the same loopback origin as its API.	Public local page
`GET /api/bridge-probe`	Unauthenticated, content-free check used to verify the dashboard origin.	Local availability metadata
`POST /api/bootstrap`	Exchange a 60-second, single-use code from the exact local origin for an in-memory bearer token.	One-time authentication setup
`GET /api/ping`	Connection status, admin flag, computer and user name.	Local identity metadata
`GET /api/health`	OS caption/build and uptime.	System metadata
`GET /api/actions`	Action registry with permission, risk, approval and audit labels.	Read-only metadata
`GET /api/projects`	Candidate roots and nearby Git repositories, paths/branch/status.	Filesystem metadata
`GET /api/stats`	CPU, RAM, disk, uptime, host and process count.	System metadata
`GET /api/processes`	Top 60 processes by memory.	Process metadata
`POST /api/process/stop`	Force-stop a PID supplied in request JSON.	**Destructive**
`GET /api/services`	Enumerate Windows service name, display name, status and startup type.	System metadata
`POST /api/service`	Start, stop or restart a named Windows service.	**System mutation**
`POST /api/action`	Run a named allowlisted action; checks registration, approval and admin flag.	Varies; some require explicit approved field
`POST /api/run`	Explicitly disabled; arbitrary PowerShell input is rejected.	**HTTP 403**
`GET /api/ollama/tags`	Proxy installed Ollama model list.	Local inference metadata
`POST /api/ollama/chat`	Proxy chat request; forces non-streaming mode for JSON response handling.	Local inference

Method + route	Purpose / data	Impact
`GET /api/mcp/status`	Return empty status while no reviewed server registry is configured.	Placeholder
`POST /api/mcp/start`	Explicitly disabled until a reviewed server registry exists.	HTTP 403
`POST /api/mcp/stop`	No managed-process lifecycle is currently connected.	Placeholder
`GET /api/claude-config`	Read Claude Desktop config JSON from the current user profile.	Potentially sensitive config
`POST /api/claude-config`	Validate JSON, make timestamped backup, then write config.	User config write

Action registry

Category	Examples	Current guard indicated in registry/route
Low-risk utilities	Lock, cancel shutdown, flush DNS, Task Manager, File Explorer, Settings, Device Manager, Windows Update.	Most are marked standard permission; no approval required.
Power changes	Sleep, restart, shutdown; performance profiles.	Sleep/restart/shutdown and high-performance profile are flagged approval-required; some require admin.
Cleanup	Empty recycle bin; clean user temp directory.	Approval-required in registry; destructive file operations.

> **Current policy:** `/api/run` and MCP startup are disabled; process, service, configuration, and high-impact action writes require explicit server-side approval. The bridge remains a powerful local trust boundary and must not be exposed to a network.

6. Security, privacy, and trust boundaries

Existing controls

- Loopback-only listener prefix limits direct access to the host machine and avoids an intentional LAN bind.
- Random startup token provides an access check for protected endpoints.
- Host, client address, and origin checks attempt to limit requests to local browser contexts.
- Allowlisted system actions include risk/permission/approval metadata, and the server checks approval/admin status for those actions.
- Claude config writes create a timestamped backup before replacing the file.
- The README warns not to publish the bridge and recommends loopback serving for the dashboard.

Risk heat map

Risk	Likelihood / impact	Why it matters	Recommended treatment
Local bridge compromise	High impact	A valid local bearer token can access sensitive workstation metadata and invoke reviewed Windows operations.	Keep loopback-only; protect browser and workstation; retain approval checks and conservative named actions.
Bootstrap link exposure	Short-lived	The browser launch URL contains a one-time setup code in its fragment for up to 60 seconds.	Fragment is removed before exchange; code expires and cannot be replayed; do not share the temporary URL.

Risk	Likelihood / impact	Why it matters	Recommended treatment
Wide Windows management routes	Medium	Process and service endpoints can mutate local state, but require explicit server-side approval; MCP startup is disabled.	Keep request schemas strict, review each named action, and test mutations only in a controlled environment.
Misleading static security indicators	Medium	Security and audit UI rows are hard-coded, not derived from event data.	Label as guidance until actual checks and durable audit events exist.
Silent dashboard failures	Medium	Some fetch/load code catches and suppresses exceptions; users may see stale/placeholder metrics.	Expose per-panel last-updated, error message, and retry states; never present stale values as fresh.
Local file discovery scope	Low	Project discovery lists the bridge's project folder and immediate child Git repositories; paths reveal local workspace layout.	Keep discovery bounded and avoid including sensitive paths in shared exports.
Public bridge exposure	Unacceptable	Machine-control API is not a hosted multi-user service; network exposure could transfer control.	Never publish the real bridge. Public demo must use synthetic backend and no machine capability.

Privacy/data handling

- Bridge responses include computer name, user name, CPU/memory/disk usage, process names, service inventory, candidate folder paths, and optionally project Git state.
- Local AI prompts go to the configured local Ollama server by default; the bridge forwards chat content and model name.
- Cloud chat sends prompt contents and an API key directly from the browser to the selected third-party provider. The cloud key remains in memory and is not persisted in browser storage.
- No code in the reviewed project establishes a telemetry or analytics system. The absence of observed telemetry is not a substitute for a documented release privacy review.

> **Safe deployment model:** keep the workstation UI/bridge private and local. If sharing a demo publicly, run a separate version with mock data and no routes to local processes, files, services, shell, models, or credentials.

7. Verification performed and evidence

This section records the checks actually performed in the available Windows environment. It does not claim comprehensive automated testing or certification across every browser or OS.

Check	Observed result	Evidence boundary
Python bridge syntax	Pass `py -3 -m py_compile bridge.py tests\test_bridge.py` completed successfully.	Current available Python 3.14 runtime; Python 3.10 minimum was not separately installed.
Dashboard syntax, navigation, routes	Pass Node check compiled the inline script and verified 12 navigation targets, 10 API routes, and 78 DOM IDs.	Static structural check, not full browser interaction.
Python bridge unit tests	Pass Six standard-library tests cover dashboard/probe serving, authentication, one-time bootstrap expiry/replay, route behavior, approvals, input limits, and parallel ping during slow chat.	Windows/Ollama operations are stubbed where appropriate; no mutation tests.

Check	Observed result	Evidence boundary
Legacy PowerShell syntax/smoke	**Pass (legacy)** Earlier parser and smoke tests passed for `bridge.ps1`.	Legacy bridge only; not evidence for the Python default runtime.
Python-served browser integration	**Pass** One-time fragment bootstrap connected automatically, removed the code from the visible URL, and kept the token out of saved browser state. All 12 sections navigated successfully.	One integrated Chromium-based browser and one local workstation.
Live Windows data routes	**Pass** Overview returned CPU/RAM/disk data; project list loaded; Processes rendered 60 rows and Services rendered 296 rows; process-name filtering worked.	Read-only checks on the current Windows machine; no process or service mutation.
Ollama integration	**Pass** 15 installed models were listed and a local chat returned `READY`; sidebar remained Connected.	One local model and a short prompt; no throughput/quality tests.
One-click batch launcher	**Pass** Double-click launcher smoke run exited successfully, selected fallback port 8789, and served dashboard (HTTP 200) and bridge probe (HTTP 204).	Python 3.14 and one Windows host; no multi-version matrix.
Terminal and MCP startup	**Disabled** Unsafe routes are rejected by policy and exercised in tests.	No arbitrary command or MCP process execution is available.
Firefox	**Not tested** Firefox executable was not present in the test environment.	No claim of Firefox support validation.

Defects found and corrected during the project work

1. **PowerShell parser failure:** malformed token comparison and duplicate authorization helper prevented the bridge script from parsing. Replaced it with one valid check and updated the request handler to use that function.
2. **Local AI empty responses:** dashboard read a single JSON object, but the bridge forwarded Ollama's default stream mode. The bridge now requests `stream = false` and returns the complete response; the UI chat was then verified.
3. **One-click startup and connection:** added a Python standard-library server, a double-click Windows launcher, and a single-use in-memory browser bootstrap credential. Browser bearer tokens are not saved to localStorage.
4. **Non-blocking API work:** the Python bridge uses bounded request threads so slow local inference does not block ping.

> **Not performed:** service/process/power mutation tests, external network exposure, adversarial security review, Firefox UI run, Python 3.10 compatibility run, and public staging deployment.

8. Known limitations and recommendations

Immediate P0 — required before broader local rollout

1. **Keep command execution disabled.** `/api/run` returns HTTP 403; do not re-enable arbitrary scripts. If named terminal tasks are added, define fixed operations, strict argument schemas, timeouts, output caps, and explicit server-side approvals.
2. **Unify authorization and policy.** Every mutating route (actions, service, process, MCP, config write, command execution) should pass a central policy function with role, target, approval, and audit metadata.
3. **Complete resource limits.** Request bodies are capped at 1 MiB and server concurrency is bounded; ensure PowerShell output limits and cancellation behavior are also consistently enforced and tested.
4. **Make errors visible.** Remove silent catches in stats/provider flows; add last-success timestamp, error message, retry, and stale-data indication to each panel.

5. **Protect browser state.** Handle invalid JSON/corrupt localStorage and browser storage denial. Avoid storing cloud API keys and long-lived bridge credentials unnecessarily; provide an explicit clear/logout flow.
6. **Review configuration endpoints.** Project discovery is limited to the project folder and immediate Git repositories; restrict Claude config read/write to an intentional UI, with backups and clear confirmation.

P1 — release quality

- Add regression tests for parser cleanliness, auth, hostile origin, malformed JSON, unknown route, invalid action, approval/admin denial, request limits, and shutdown cleanup.
- Exercise the supported browser versions on Windows at desktop and narrow viewport sizes; record actual version, result, and screenshot/test evidence.
- Implement keyboard-visible focus, accessible labels, reduced-motion support, error announcements, responsive table handling, and a deliberate light-theme palette.
- Distinguish actual tooling discovery from static “Available locally” claims.
- Add real audit events with timestamps, action, target, caller context, decision and result; avoid recording secrets or full command content where unnecessary.

P2/P3 — product expansion

- Explicit project folder selection, safe file tree, manifest/language detection, secret-aware excludes, searchable local index, and source-referenced answers.
- Patch-based editor workflow with diff review, per-file approval, backups, apply/reject/revert and verified test outcomes.
- Ollama health state, streaming with cancellation, history persistence controls, model settings, and timeouts.
- Wire MCP server lifecycle UI to bridge routes only after executable allowlisting and isolation are specified.
- Create a separate public demo mode with synthetic data. Do not tunnel or forward the live local bridge.

9. Upgrade roadmap and acceptance gates

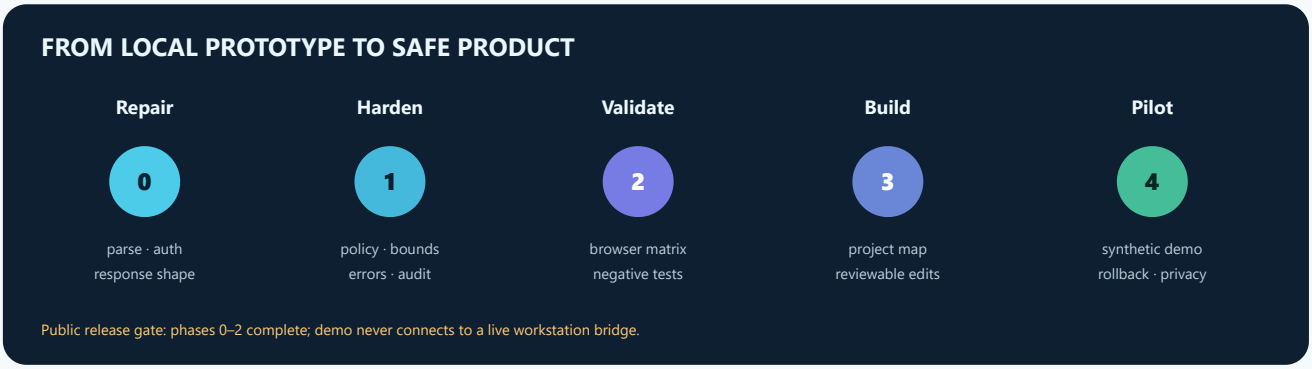


Figure 3. Sequencing avoids building feature breadth on top of an unsafe or untestable execution foundation.

Phase	Deliverables	Acceptance gate	Priority
0 · Stabilize	Parser/auth/chat fixes, startup steps, repeatable smoke test.	Parse clean; valid/invalid auth covered; live local chat works; known error states documented.	P0 · in progress / partly verified
1 · Secure	Central policy, restricted commands, input/output/time bounds, error reporting, audit log, storage cleanup.	Mutation routes have server-side policy; rejected requests do not alter machine state; audit evidence exists.	P0
2 · Qualify	Browser/device matrix, accessibility and responsive pass, install/stop docs, regression automation.	Supported browser list and recorded test evidence; no blocker defects in representative offline/connected cases.	P1

Phase	Deliverables	Acceptance gate	Priority
3 · Workspace	Explicit allowed folders, safe project explorer, metadata index, model-assisted cited context.	Traversal and secret-exclusion tests pass; user can clear context/history.	P2
4 · Edit safely	Diff-based patches, approvals, backup/rollback, controlled test execution.	Rejected patch never writes; accepted patch is recoverable; test exit status is accurate.	P2
5 · Public pilot	Isolated mock deployment, privacy statement, support/feedback, rollback, release notes.	No bridge credentials or workstation endpoint in demo; synthetic-data tests and rollback pass.	P3

> **Go/no-go:** no public launch of the machine-control bridge. Approve a public-facing pilot only after P0/P1 gates and only against a separate mock or sandbox service.

10. Installation, startup, and daily-use guide

Prerequisites

- Windows 10/11 with Python 3.10 or newer; Windows PowerShell 5.1 is invoked for system operations.
- A current browser. The UI was exercised in the VS Code integrated Chromium browser; other browser products should be tested directly before claiming support.
- No pip packages or separate web server are required; the Python standard-library bridge serves the HTML dashboard and API.
- Optional: Ollama reachable at `http://127.0.0.1:11434` with at least one pulled model.

Start the bridge

Double-click `Start-Helpyard.bat`, or start from PowerShell with:

```
py -3 .\bridge.py --open-browser
```

The launcher starts a minimized bridge console. Keep it open while using the dashboard; close it or press `Ctrl+C` there to stop the service. The bridge does not print its bearer token.

Open the dashboard

The launcher opens a temporary connection URL in the default browser. The browser consumes a short-lived one-time code from its fragment, removes the fragment, and keeps the returned bearer token in memory. Dashboard and API share one loopback origin even when a fallback port is selected.

Connect and operate

1. Open Settings from the sidebar.
2. The Bridge URL is detected from the Dashboard URL; verify it matches the current bridge run.
3. The browser connects automatically and the sidebar should show Connected and the host name. Manual token storage is not required.
4. Use Overview first; navigate to Projects, Processes, Services, or Local AI only as needed. Refresh buttons reload related data.
5. Use Terminal and action buttons only for commands/actions understood and intentionally authorized. A browser confirmation does not sandbox or neutralize the command.

6. Clear saved browser data from Settings when finished on a shared profile. Stop the bridge when done.

Common recovery

Symptom	Check	Response
Sidebar stays Offline	Bridge process is still running; browser opened from the current launcher run; connection code has not expired.	Close the bridge and double-click `Start-Helpyard.bat` again to create a fresh one-time session.
Ollama Offline / no models	Ollama service and model list endpoint.	Start Ollama, pull a model, refresh Local AI; dashboard remains usable without AI.
Administrator-required action rejected	Bridge says the current process is not elevated.	Use the standard account for routine work. Elevate only if the specific action is required and understood.
Port conflict	Bridge startup output reports selected fallback port.	Open the printed Dashboard URL; the Bridge URL is detected automatically.
Browser cannot fetch bridge	Verify the bridge terminal is still running and the Dashboard URL is from that run.	Restart the bridge and open its new Dashboard URL; avoid remote origins and do not broaden bridge binding.

11. Project files and glossary

Reviewed and generated files

File	Role	Report note
`pc-control-dashboard.html`	Application interface, styling and browser logic.	Single-file dashboard; main user entry point.
`bridge.py`	Default Windows localhost HTTP service and API routes; invokes fixed Windows PowerShell scripts.	Privileged trust boundary; must not be public-hosted.
`Start-Helpyard.bat`	One-click Python version check and minimized bridge/browser startup.	Requires Python 3.10+ installed for Windows.
`bridge.ps1`	Legacy PowerShell bridge retained for manual use and its separate smoke test.	Not used by the default launcher.
`tests\test_bridge.py`	Standard-library Python API and security regression tests.	Uses stubs for Windows/Ollama operations; see README for scope.
`README.md`	Startup, safety and validation guidance.	Updated to reflect regular-user startup and loopback serving.
`update plan no1.txt`	Original product vision and broad implementation ideas.	Describes future project intelligence, editor and 3D workspace goals; those goals are not all implemented.
`Plannumber1.Pdf`	Numbered upgrade and launch plan.	Prior companion deliverable; execution gates and phase plan.
`Plannumber1.html`	Editable HTML source for the prior plan PDF.	Print-ready source for revision.

File	Role	Report note
`Project Report Number1.html`	Editable source for this report.	Contains vectors/diagrams and presentation styling.
`Project Report Number1.Pdf`	Requested presentation/report artifact.	Generated from this HTML source for sharing/printing.

Glossary

Term	Meaning in this project
Bridge	The PowerShell process that exposes a local HTTP API to the browser.
Loopback	Local host-only network interface, typically 127.0.0.1; not intended for remote users.
Bearer token	Secret string sent in the Authorization header to authenticate bridge requests.
Ollama	Optional local service for listing and running installed language models.
MCP	Model Context Protocol; a potential integration area, only partly represented in the current interface.
LocalStorage	Browser-origin key/value storage; convenient but not equivalent to a secure credential vault.
Mock backend	Safe substitute that returns synthetic data without reaching a user's real workstation.

Final assessment

Helpyard has a compact, understandable architecture and demonstrates useful local workstation workflows. The live system/bridge and local AI path were verified in one Windows Chromium-based environment. Its next success criterion is not more screens; it is consistently enforcing least privilege, surfacing errors accurately, and capturing repeatable cross-browser evidence. Treat the current build as a local prototype until those controls and release gates are completed.